

LAB9: Computer Vision

Object Detection and Localization using YOLO in Computer Vision

Objective:

- To implement object detection using deep learning models.
- To detect and localize multiple objects in images using YOLO.
- To analyze detection performance on images and real-time data.

LAB: Computer Vision

Description

-Description Carefully observe the key steps for object detection

- *Understand difference between classification and detection:*
- *Classification → one label*
- *Detection → multiple objects + bounding boxes*
- *YOLO detects objects in a single forward pass, making it fast and efficient*

Install Required Libraries

```
[1]: pip install ultralytics opencv-python matplotlib
```

Collecting ultralytics

Downloading ultralytics-8.4.39-py3-none-any.whl.metadata (39 kB)

Requirement already satisfied: opencv-python in c:\users\hp\py310_env\lib\site-packages (4.13.0.90)

Requirement already satisfied: matplotlib in c:\users\hp\py310_env\lib\site-packages (3.10.8)

Requirement already satisfied: numpy>=1.23.0 in c:\users\hp\py310_env\lib\site-packages (from ultralytics)

Requirement already satisfied: pillow>=7.1.2 in c:\users\hp\py310_env\lib\site-packages (from ultralytics)

Requirement already satisfied: pyyaml>=5.3.1 in c:\users\hp\py310_env\lib\site-packages (from ultralytics)

```
[2]: import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

Load Pretrained YOLO Model & Pass Image (Multiple objects)

```
[3]: model = YOLO("yolov8n.pt")    # lightweight model
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100%  
0.1ss3.0s
```

```
[5]: image_path = "imageM.png"    # use your own image
```

```
results = model(image_path)
```

```
# Show detection results
```

```
results[0].show()
```

Perform Object Detection on Image

```
[5]: image_path = "imageM.png"    # use your own image
      results = model(image_path)

      # Show detection results
      results[0].show()
```

Display Image

```
img = cv2.imread(image_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img)
plt.axis('off')
plt.title("Input Image")
plt.show()
```

Input Image



Extract Detection Details

```
[7]: for r in results:
      for box in r.bboxes:
          print("Class:", box.cls)
          print("Confidence:", box.conf)
          print("Coordinates:", box.xyxy)
```

```
Class: tensor([2.])
Confidence: tensor([0.9106])
Coordinates: tensor([[573.2513, 496.6302, 797.5087, 674.5331]])
Class: tensor([0.])
Confidence: tensor([0.8927])
Coordinates: tensor([[248.8316, 472.4067, 434.2791, 901.6660]])
Class: tensor([0.])
Confidence: tensor([0.8846])
Coordinates: tensor([[834.4497, 475.0217, 977.5854, 860.4755]])
Class: tensor([0.])
Confidence: tensor([0.8740])
```

Extract Detection Details

```
[7]: for r in results:
      for box in r.bboxes:
          print("Class:", box.cls)
          print("Confidence:", box.conf)
          print("Coordinates:", box.xyxy)
```

```
Class: tensor([2.])
Confidence: tensor([0.9106])
Coordinates: tensor([[573.2513, 496.6302, 797.5087, 674.5331]])
Class: tensor([0.])
Confidence: tensor([0.8927])
Coordinates: tensor([[248.8316, 472.4067, 434.2791, 901.6660]])
Class: tensor([0.])
Confidence: tensor([0.8846])
Coordinates: tensor([[834.4497, 475.0217, 977.5854, 860.4755]])
Class: tensor([0.])
Confidence: tensor([0.8740])
```


Tasks for Students

1. Display 3 images with detected objects and labels
2. Run object detection and report:
3. Number of detected objects
4. Confidence scores
5. Modify-Use a different YOLO model and write obs

- *At the end create pdf file and submit, don't be late, late submission may affect your sessional marks*